

Detalhamento técnico

Documento exigido pelo desafio. As seções **1 a 7** respondem item a item o que a especificação pede, com o motivo de cada escolha — não só o que foi usado, mas por que valia a pena. As seções **8 a 11** cobrem o que o desafio não perguntou e sustenta boa parte do comportamento do sistema: a mensageria, o provedor de IA, os testes e a segurança.

Autor: Thiago Dias · Repositório: https://github.com/thiago-dsd/Korp_Teste_ThiagoDias

1. Ciclos de vida do Angular utilizados

O Angular 22 permite resolver com *signals* e injeção boa parte do que antes exigia um ciclo de vida. Por isso os que aparecem no código são poucos e cada um tem uma razão concreta.

CICLO DE VIDA	ONDE	POR QUÊ
<code>ngOnInit</code>	telas de produtos, notas, detalhe da nota, dashboard, formulários, painel de histórico	Ponto em que os <i>streams</i> de busca e recarga são montados. Não fica no construtor porque montar assinaturas ali dificulta o teste e roda antes de os <code>input()</code> estarem disponíveis: um <code>input.required</code> lido no construtor ainda não tem valor, e o Angular responde com <code>NG0950</code> .
<code>ngOnDestroy</code>	<code>ClickOutsideDirective</code> , <code>MenuService</code> , <code>ModalComponent</code>	Os três guardam recursos que o Angular não recolhe sozinho: um ouvinte de evento no <code>document</code> , uma assinatura de longa duração e, no diálogo, a rolagem travada do <code>body</code> e o foco que precisa voltar para onde estava.
<code>ngAfterViewInit</code>	<code>ClickOutsideDirective</code> , <code>NavbarSubmenuComponent</code> , <code>ModalComponent</code>	Precisam do elemento já renderizado para medir, ouvir ou receber foco.

O que substituiu os demais, e é a parte que merece atenção:

- `takeUntilDestroyed(destroyRef)` (11 arquivos) elimina o par `ngOnDestroy` + `Subject` + `takeUntil` que normalmente aparece em toda tela. A assinatura morre junto com o componente sem que exista um método para isso.
- `effect()` carrega o produto no formulário quando o `input()` muda — o papel que antes era do `ngOnChanges`, mas reagindo ao valor e não ao ciclo de renderização.
- `computed()` (13 arquivos) deriva estado (`hasMore`, `selectionFull`, `pendingAdjustments`, `isAdmin`) em vez de recalcular em `ngDoCheck` ou no template.

- `input()` / `output()` substituem `@Input` / `@Output`; `output()` aparece em `product-form` (`save`, `cancelled`), `bulk-result` (`dismiss`), `modal` e `product-movements` (`closed`).
- `inject()` (39 arquivos) no lugar de injeção por construtor.

A aplicação roda **zoneless** (`provideZonelessChangeDetection()` em `main.ts`): sem Zone.js, a detecção de mudanças é disparada pelos próprios signals, o que torna `OnPush` desnecessário na maioria dos componentes — ele aparece só onde há um motivo explícito, como no `app-bulk-result`.

Isso tem uma consequência prática que apareceu nos testes: `fakeAsync` deixa de funcionar sem Zone.js, e por isso o intervalo de *polling* da impressão é injetável (`PRINT_POLL_INTERVAL`) em vez de manipulado por relógio falso.

2. Uso de RxJS

Sim, e de forma deliberada: **signals para estado, RxJS para eventos ao longo do tempo**. Onde só existe estado, não há Observable; onde existe uma sequência de acontecimentos, RxJS resolve com menos código do que qualquer alternativa.

Operadores em uso: `debounceTime`, `distinctUntilChanged`, `switchMap`, `startWith`, `catchError`, `tap`, `map`, `of`, `throwError`, `finalize`, `shareReplay`, `takeWhile`, `timer`, `forkJoin`, `fromEvent`, `filter`, `Subscription`, `toSignal`, `takeUntilDestroyed`.

Os três usos que justificam a biblioteca:

Busca no catálogo (`products.component.ts`) — `debounceTime(300)` espera a pessoa parar de digitar, `distinctUntilChanged()` descarta o termo repetido e `switchMap` **cancela a resposta de uma busca que já foi substituída por outra mais nova**. Sem esse último, uma resposta lenta sobrescreveria na tela o resultado de uma busca posterior — o clássico problema de corrida de digitação.

Acompanhar a impressão (`invoice.service.ts`) — a impressão é assíncrona, então `timer(intervalo, intervalo)` + `switchMap(() => this.get(id))` + `takeWhile(status === 'PRINTING', true)` consulta a nota até ela sair de PRINTING e **completa entregando a nota final** como última emissão. O `true` no `takeWhile` é o detalhe que faz o resultado final chegar ao assinante.

Filtros vindos da URL (`products.component.ts`, `invoices.component.ts`) — a query string é a fonte da verdade do que está na tela. `route.queryParamMap` → `map` para um estado → `distinctUntilChanged` → `switchMap(() => this.fetch())`. Os controles não recarregam a lista: eles navegam, e a navegação volta por esse mesmo *stream*. O `distinctUntilChanged` é o que impede o laço, já que escrever na URL reemite o parâmetro.

Painéis independentes do dashboard (`home.component.ts`) — `forkJoin` de quatro listagens, cada uma com o seu próprio `catchError` que degrada para página vazia. O `catchError` fica **dentro** do `forkJoin`, não fora: fora dele, um serviço lento derrubaria os quatro painéis e a tela ficaria em branco em vez de mostrar o que deu para ler.

Renovação de sessão (`auth.service.ts` + `auth.interceptor.ts`) — quando o token expira com várias requisições em voo, todas precisam esperar **uma única** renovação. O `AuthService` mantém a renovação em curso com `shareReplay(1)` e o interceptor apenas encadeia a requisição original com `switchMap`

depois dela. Sem isso seriam N renovações concorrentes — e, com detecção de reuso de refresh token no servidor, isso derrubaria a sessão inteira.

3. Outras bibliotecas e suas finalidades

Frontend

BIBLIOTECA	FINALIDADE
Angular 22	Framework da SPA: componentes <i>standalone</i> , roteamento com <i>lazy loading</i> , formulários reativos tipados, <code>HttpClient</code> com interceptors.
Tailwind CSS 4	Estilos utilitários direto no template, com <code>@tailwindcss/forms</code> (campos), <code>@tailwindcss/typography</code> , <code>@tailwindcss/aspect-ratio</code> e <code>tailwind-scrollbar</code> .
angular-svg-icon	Ícones SVG inline (Heroicons), permitindo colorir por CSS.
ngx-sonner	Notificações <i>toast</i> de confirmação e erro.
Vitest (via <code>@angular/build:unit-test</code>) + jsdom	Testes unitários do frontend — 218 no total.
Playwright	Testes end-to-end dos fluxos de produto e nota fiscal.
Prettier	Formatação, verificada no CI.

Backend

BIBLIOTECA	FINALIDADE
jackc/pgx v5	Driver PostgreSQL e <i>pool</i> de conexões. Escolhido por expor recursos que o <code>database/sql</code> esconde e que este projeto usa: <code>FOR UPDATE SKIP LOCKED</code> , <code>COPY / unnest</code> com arrays, savepoints e códigos de erro do Postgres (violação de unicidade tratada por código, não por texto).
golang-jwt/jwt v5	Assinatura e verificação RS256 dos tokens de acesso.
rabbitmq/amqp091-go	Cliente AMQP: <i>publisher confirms</i> , <i>dead letter exchange</i> , reconexão.
google/uuid	Identificadores.
golang.org/x/crypto	<code>argon2id</code> para hash de senha.

São **cinco dependências diretas**. Tudo o mais — HTTP, JSON, roteamento, testes, logging estruturado (`log/slog`), concorrência — vem da biblioteca padrão.

Bibliotecas de componentes visuais

Nenhuma. Não há Angular Material, PrimeNG ou similar. A interface é construída com Tailwind e componentes próprios (`app-modal`, `app-bulk-result`, `app-invoice-status`, `app-stock-level`, `app-product-form`, `app-product-import`, `app-product-movements`, `app-language-switcher`).

O que uma biblioteca costuma dar de graça está resolvido em dois lugares e não espalhado pelos *templates*: `app-modal` concentra o que faz um diálogo ser um diálogo (Escape, foco inicial, foco devolvido ao fechar, rolagem travada atrás), e um punhado de *utilities* `action-*` em `styles.css` concentra cor e estado de cada papel de ação — sólida, destrutiva, contornada, alternador ligado e desligado, ação só-texto —, deixando no elemento apenas tamanho e forma.

O motivo é específico deste sistema: as telas são poucas e o comportamento delas é o que importa — seleção que sobrevive à paginação, painel que mostra sucesso parcial, formulário que preserva o que foi digitado quando o servidor recusa. Uma biblioteca de componentes resolve a aparência, que aqui foi a parte barata, e não esses comportamentos, que foram onde o trabalho esteve.

4. Gerenciamento de dependências no Go

Go Modules, que é o mecanismo oficial desde o Go 1.11.

- `go.mod` declara o módulo (`github.com/thiagodias/korp-invoices`), a versão da linguagem (1.26.4) e as versões exatas de cada dependência.
- `go.sum` guarda o *hash* criptográfico de cada módulo. Uma dependência que mude de conteúdo sob a mesma versão faz o build **falhar**, o que é a defesa contra um pacote adulterado no meio do caminho.
- `go mod tidy` mantém o arquivo alinhado ao que o código realmente importa; dependências diretas e indiretas ficam separadas em blocos.
- **Um único módulo** para os três serviços. Eles compartilham `internal/platform` e `internal/contracts` — os contratos de evento existem justamente para que os dois lados compilem contra a mesma definição, e módulos separados transformariam uma mudança de payload em uma publicação de versão.
- `internal/` impede que qualquer código fora do módulo importe esses pacotes: é o próprio compilador garantindo a fronteira.
- Não há `vendor/`: o `go.sum` já garante reprodutibilidade, e o cache de módulos é preservado entre execuções no CI e na camada de build do Dockerfile.

5. Frameworks utilizados no Go

Nenhum framework web. O roteamento é o `net/http` da biblioteca padrão, usando o suporte a método + caminho introduzido no Go 1.22:

```

mux.Handle("POST /products", guard(admin(write(http.HandlerFunc(a.createProduct)))))
mux.Handle("GET /products/{id}", guard(read(http.HandlerFunc(a.getProduct))))

```

Foi uma decisão, não uma omissão. Um framework como Gin ou Echo entregaria roteamento com parâmetros, *middlewares* encadeados e *binding* de JSON — e a biblioteca padrão passou a fazer as três coisas. O que este projeto realmente precisava, nenhum deles daria pronto: outbox transacional, idempotência com replay, cerca entre tentativas de impressão, limitação por categoria de operação. Adicionar um framework significaria uma dependência a mais no caminho de toda requisição para resolver o que a biblioteca padrão já resolve.

Os *middlewares* são funções `func(http.Handler) http.Handler` compostas por `httpx.Chain`: request id → log → recover → CORS → timeout → métricas → limite público → idempotência, e, por rota, autenticação → papel → limite da categoria.

Também não há ORM. O SQL é escrito à mão porque as consultas que importam aqui não são as que um ORM gera bem: `UPDATE ... WHERE balance >= $2` que recusa saldo negativo no próprio banco, `FOR UPDATE SKIP LOCKED` no outbox, `ON CONFLICT (consumer, message_id) DO NOTHING` para reconhecer uma mensagem reentregue, paginação por *keyset*, `unnest` para inserir os itens de uma nota em uma ida ao banco.

Migrações são versionadas e aplicadas pelo próprio serviço no *start*, com *advisory lock* (só uma instância migra) e uma transação por migração.

6. Tratamento de erros e exceções no backend

Go não tem exceções: erros são valores e viajam explicitamente. O projeto padroniza isso em um vocabulário próprio, `internal/platform/apperr`.

Um erro carrega o que é preciso para agir

```

type Error struct {
    Kind    Kind           // invalid, not_found, conflict, unauthorized,
                                // forbidden, too_many_requests, unavailable, internal
    Code    string         // "insufficient_balance" – estável, para o cliente
    Message string         // escrito para o operador, não para o log
    Details map[string]string // campo → o que há de errado
    Cause   error          // o erro original, para o log
}

```

A tradução para HTTP acontece **em um lugar só** (`httpx.StatusFor`), a partir do `Kind`. Nenhum handler escreve `http.StatusConflict`.

Decisões que valem explicar

- **Kind separado de Code.** O `Kind` diz o que o transporte deve fazer; o `Code` diz o que aconteceu. `forbidden` e `unauthorized` são *kinds* distintos de propósito: entrar de novo resolve um e não resolve o outro.
- **Sentinelas comparáveis.** `apperr.Error.Is` compara `Kind` + `Code`, então `errors.Is(err, stock.ErrDuplicatedCode)` continua funcionando depois de `WithCause` / `WithDetails`. Enriquecer um erro com causa ou detalhes não quebra quem o compara mais acima.
- **Validação acumulada.** Um corpo inválido responde com **todos** os campos ofensores de uma vez em `Details`, não com o primeiro que falhou.
- **A causa nunca vaza.** `Cause` vai para o log; o cliente recebe `Message` e `Code`. Um erro interno responde "An unexpected error occurred." com o `request_id` para correlação.
- **Pânico não derruba o processo.** O middleware `Recover` converte em 500 e registra.
- **Erro permanente x transitório.** No consumo de mensagens, `resilience.Permanent(err)` marca o que não adianta repetir (payload malformatado) e interrompe o retry na hora; o resto é repetido com *backoff* e, esgotado, vai para a *dead letter queue* — que é inspecionável e reprocessável por endpoint interno.
- **Rejeição de negócio não é falha.** No débito de estoque, saldo insuficiente é **uma resposta**: vira o evento `stock.rejected` e a nota reabre com o motivo. Só erro de infraestrutura é repetido.
- **Erros embrulhados com %w** preservam a cadeia; o contexto é adicionado na borda de cada camada (`fmt.Errorf("insert invoice item: %w", err)`).

Do lado do cliente

O `apiErrorInterceptor` transforma toda falha em `ApiError` com `code`, `message`, `details` e `status`, e as telas mostram a `message` — que já foi escrita para quem está lendo. Serviço fora do ar vira `service_unreachable` em vez de um erro de rede cru.

7. LINQ / C#

Não se aplica: o backend é em **Go**.

8. Mensageria: por que existe, e por que RabbitMQ

O problema que ela resolve

Imprimir uma nota toca **dois serviços com dois bancos**: o faturamento fecha a nota, o estoque debita os saldos. Não existe transação distribuída entre eles, então uma chamada HTTP síncrona deixaria duas perguntas sem resposta boa: o que acontece se o estoque estiver fora do ar no momento do clique, e o que acontece se ele debitar mas a resposta se perder no caminho.

A escolha aqui foi tornar o pedido **durável antes de ser entregue**.

Padrão outbox: estado e mensagem, ou nenhum dos dois

`internal/billing/print_store.go` grava a mudança de status e a mensagem `invoice.print_requested` na **mesma transação**. Ou as duas coisas acontecem, ou nenhuma. Não existe o estado "a nota foi fechada mas o estoque nunca ficou sabendo", nem o contrário.

Um *relay* (`messaging/relay.go`) roda dentro do serviço, lê o que está na `outbox_messages` e publica no broker. Se o broker estiver fora, a mensagem continua na tabela e sai quando ele voltar — e o relay avisa no log quando a fila para de escoar, em vez de reclamar uma vez por mensagem.

É isso que o vídeo demonstra ao derrubar o estoque no meio de uma impressão: a nota fica em `PRINTING`, o pedido espera na outbox, e quando o serviço volta ela fecha sozinha.

Entrega ao menos uma vez, efeito exatamente uma vez

Um broker que garante entrega acaba entregando de novo em caso de dúvida. Quem consome grava em `processed_messages`, **na mesma transação do trabalho**, que já tratou aquela mensagem (`messaging/inbox.go`). A redundância é reconhecida e descartada: entrega ao menos uma vez vira efeito exatamente uma vez, que é o que importa para um débito de saldo.

Por que RabbitMQ e não Kafka

O trabalho aqui é **distribuir tarefas**, não guardar um histórico de eventos para reprocessar depois. O que este sistema precisa é confirmação por mensagem, redelivery, e um lugar para onde mandar o que não pôde ser processado — que é exatamente o forte do RabbitMQ:

RECURSO USADO	ONDE
<i>Publisher confirms</i>	o relay só apaga da outbox depois que o broker confirmou
Filas duráveis	sobrevivem ao restart do broker
<i>Dead letter exchange</i> (<code>invoices.dlx</code>)	mensagem que falhou 3 tentativas com <i>backoff</i> vai para a <code>.dlx</code> , em vez de sumir
Endpoint de <i>replay</i>	<code>POST /internal/dead-letters/replay</code> devolve as mensagens mortas à fila, depois de corrigida a causa
<code>Qos /prefetch</code>	limita quanto um consumidor puxa de uma vez

O Kafka é forte no que este sistema **não** faz: retenção longa, releitura por *offset*, vazão alta particionada. Em troca, custa mais para operar. Aqui ele seria complexidade sem uso.

9. IA: por que Azure AI Foundry

O sistema não depende dele

`internal/platform/aiclient` existe para que o resto do código dependa da **interface `Model`**, e nunca da implementação. Trocar de provedor é escrever outro arquivo nesse pacote; nenhuma regra de negócio

muda. Os dois assistentes (rascunho e busca) compartilham o mesmo *deployment* pela mesma interface.

As vantagens que pesaram

Fica dentro da assinatura da empresa. O *deployment* vive numa subscription Azure, com região, quota e faturamento próprios — não numa conta pessoal de terceiro com uma chave avulsa. Para um sistema que lida com nota fiscal, onde o dado sai da rede, sob que contrato e em que região são perguntas que aparecem cedo.

A API clássica é HTTP puro. `POST {endpoint}/openai/deployments/{deployment}/chat/completions` com o header `api-key`. Sem SDK: são ~200 linhas que controlamos, com *timeout* de 20s, política de *retry* que distingue erro transitório de rejeição, e teto no tamanho da resposta lida. Uma dependência a menos, e nenhuma surpresa escondida.

O custo é limitado por configuração, não por esperança. A quota é definida por *deployment* em tokens por minuto, e a versão do modelo é fixada. Somando aos limites do lado da aplicação — texto de entrada com teto, catálogo no prompt limitado, resposta limitada, e a chamada contando no *rate limit* do usuário — o gasto tem um teto conhecido.

Desligado, o sistema continua inteiro. Sem as variáveis de ambiente, `GET /invoices/draft` responde `available:false`, as telas não oferecem os campos, e nada mais degrada. A IA é um acréscimo, nunca um caminho crítico.

O README traz os comandos `az` para provisionar o mínimo que funciona.

10. Testes

429 funções de teste em 20 pacotes Go e 218 testes no frontend, rodando no CI a cada push.

O backend testa contra Postgres de verdade, não contra mock de repositório: as consultas que importam aqui são justamente as que um mock não exercita — o débito condicional que recusa saldo negativo, o `FOR UPDATE SKIP LOCKED` do outbox, a paginação por *keyset*. Os cenários de concorrência sobem goroutines disputando o mesmo saldo — `TestConcurrentDebitsOfTheLastUnit` afirma que o saldo termina em zero e nunca negativo.

No frontend são 38 arquivos de *spec*. Os das telas e serviços usam `HttpTestingController` para afirmar as três coisas que importam: o que é pedido ao serviço, o que aparece na tela e o que acontece quando o serviço recusa. O caminho de erro é coberto junto com o de sucesso — servidor fora do ar, requisição rejeitada, sucesso parcial de uma operação em lote.

11. Segurança

- **Senhas** com `argon2id`, e a verificação roda mesmo quando a conta não existe, para que o tempo de resposta não revele quais e-mails estão cadastrados.

- **Tokens de acesso** assinados em RS256. Os outros serviços verificam pela chave pública via JWKS, sem perguntar ao serviço de identidade a cada requisição.
 - **Refresh tokens** guardados como *hash* e rotacionados a cada uso. Reapresentar um já trocado revoga a sessão inteira — é o que limita o estrago quando um token vaza.
 - **Papel dentro do token assinado**, verificado por rota (`RequireRole`), com `403` para quem está autenticado e não pode, e `401` para quem não está: entrar de novo resolve um e não resolve o outro.
 - **Limite de requisições por categoria** — leitura, escrita, lote e IA têm orçamentos separados, para que ler uma nota enquanto ela imprime não gaste o mesmo saldo de emitir notas.
 - **Entrada validada na borda**, com corpo limitado em tamanho, campos desconhecidos recusados e todos os campos inválidos reportados de uma vez.
 - **CORS** com origem declarada por configuração, não `*`.
-